Compiling Facts into Applications

Samuel Lippert
Drivly Inc

Abstract

Domain knowledge enters software as natural language requirements, is re-encoded as schemas, validation logic, state machines, and API routes, and drifts out of sync in production. The informal layer between specification and execution is where bugs live and where automated agents hallucinate. We show that a single set of FORML 2 declarations can define a database schema, constraint rules, state machine workflows, and a REST API with hypermedia navigation, producing a complete application from domain facts alone. We formalize this as AREST, a Backus AST system [1] whose definitions are compiled FORML 2 readings [5] and whose state is Codd's named set [2] of ORM 2 elementary facts. REST endpoints are applications of a single system function. Ingesting new readings is self-modification. Five theorems establish that the grammar is unambiguous, that specifications and executables are the same object, that state transfer is complete, that HATEOAS links are projections of the population, and that every value in the API is derived from facts. Together, they eliminate the informal layer entirely.

1 Introduction

Software begins as domain knowledge in someone's head. It is re-encoded as schemas, validation logic, state machines, and API routes. When these are distinct artifacts, their equivalence is not preserved by construction.

AREST identifies what was never separate to begin with. A FORML 2 reading is simultaneously a relation schema, a constraint specification, a resource definition, and an FFP object. The compiler does not translate between representations. It recognizes a single object that already occupies all four roles.

2 Terminology

Three terms carry overloaded abbreviations. *AST* follows Backus's usage (Applicative State Transition, 1978), not the compiler term. *ORM* follows Halpin's usage (Object-Role Modeling, 2008), not the persistence pattern. *FP* follows Backus's usage (Function-level Programming), not the broader tradition. *FFP* is Backus's extension of FP with named definitions and a representation function ρ that maps objects to the functions they denote [1, Sec. 13].

3 Definition

Definition 1 (AREST)**.** An *Applicative REpresentational State Transfer* system is an AST system [1, Sec. 14] whose state D satisfies:

- The *FILE* cell contains a population $P = \uparrow FILE : D$, a named set of relations [2, Sec. 1.6] whose elements are ORM 2 elementary facts [5, Ch. 3]
- The *DEFS* cells contain FFP objects [1, Sec. 13.2]: readings compiled from FORML 2 declarations [5], and functions registered by the runtime

The *schema* S is the set of compiled objects in *DEFS*: fact type constructors, constraint objects, derivation rules, and state machine definitions. The representation function ρ [1, Sec. 13.3.2] maps an object to the function it represents. Given a fact type, ρ looks it up in *DEFS* and returns the functional form. The system function $SYSTEM : x = \langle o, D' \rangle$ [1, Sec. 14.3.1] is defined by the definitions in D. REST endpoints are applications of *SYSTEM* with distinct inputs.

4 Example

An order domain is declared in FORML 2:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order was placed by Customer.
  Each Order was placed by exactly one
  Customer.
Status 'In Cart' is initial in
  State Machine Definition 'Order'.
Transition 'place' is from
  Status 'In Cart'.
Transition 'place' is to
  Status 'Placed'.
Transition 'ship' is from
  Status 'Placed'.
Transition 'ship' is to
  Status 'Shipped'.
```

These nine readings compile to a database table, a foreign key, a uniqueness constraint, and a three-state machine, which is everything needed to run an order pipeline. Consider the constraint "Each Order was placed by exactly one Customer." The compiler recognizes this as a uniqueness restriction on the *placedBy* role and emits:

$$(\rho\, reading) : P_{placedBy} = Filter(count(s_1) > 1) : P_{placedBy}$$

The result is the set of Orders with more than one Customer in *placedBy*. An empty set means the constraint holds.

`POST /orders {"customer":"acme"}` applies $SYSTEM : x$ where the addressed entity is *orders* and the operation is *create*. The pipeline produces:

$$P' = P \cup \{Order(\texttt{ord-1}),\ placedBy(\texttt{ord-1}, \texttt{acme}),\ SM(\texttt{ord-1}, \texttt{In Cart})\}$$

The response includes links derived from P':

```
{ "id": "ord-1", "customer": "acme",
  "status": "In Cart",
  "_links": {
    "place": {
      "href": "/orders/ord-1/transition",
      "method": "POST" } } }
```

Following "place" advances the state machine. The new representation contains $\{ship\}$, not $\{place\}$, because the status is now Placed. If a second command assigns a different Customer to the same Order, the restriction produces a non-empty violation set and the command is rejected. The violation is returned as the original reading.

The constraint, the state machine, the API response, and the HATEOAS links above are different views of the same data. The mechanism that makes this possible is metacomposition. A fact type [5, Ch. 3] is a constructor of roles [1, Sec. 11.2.4]. Populate the roles with instances and you get a fact: $\langle ft, o_1, \ldots, o_n \rangle$. Metacomposition [1, Sec. 13.3.2] resolves a fact by looking up its fact type:

$$(\rho \langle x_1, \ldots, x_n \rangle) : y = (\rho\, x_1) : \langle \langle x_1, \ldots, x_n \rangle, y \rangle \tag{1}$$

The fact is a function: $\lambda f.\, f(o_1, \ldots, o_n)$, accepting an operation and applying it to its own objects. The constraint reading, the state machine transition, and the API response above are all ρ-applications over the same facts.

5 The System Function

Each input x addresses an entity and carries an operation. The system function routes the input to the addressed entity and passes the operation as the operand:

$$SYSTEM : x = (\rho\, (\uparrow entity(x) : D)) \uparrow op(x) \tag{2}$$

The entity is fetched from D. ρ resolves it to a functional form. The operation is applied as the operand. The entity handles the dispatch, not the system function. New operations are registered in *DEFS* without modifying any entity, and new entity types are added without modifying any operation.

Remark (Specialization of Backus's subsystem). Backus's subsystem [1, Sec. 14.4.2] dispatches on a *KEY* cell with five clauses. AREST collapses them into a single ρ-dispatch (2): the entity determines the functional form, and the HTTP method determines the operation class. Self-modification is the same mechanism: the addressed entity is *DEFS* and the operation is $compile \circ parse$. The new FFP objects are stored via $\downarrow DEFS$, and subsequent applications of *SYSTEM* evaluate the new definitions.

5.1 Commands

μ denotes evaluation [1, Sec. 14.3.1]. Each application is an AST transition: the state before, the state after, nothing else changes.

A CREATE command:

$$create = emit \circ validate \circ derive \circ resolve \tag{3}$$

resolve determines identity from the reference scheme [5, Ch. 5]. *derive* forward-chains derivation rules to the least fixed point. *validate* applies every constraint as a restriction over the population, producing violation sets. *emit* always runs. An alethic violation rejects ($D' = D$). A deontic violation warns but succeeds ($D' = D''$). The violation message is the original reading (Corollary 1).

A state machine processes events chronologically:

$$machine(s_0, E) = foldl\ transition\ s_0\ E \tag{4}$$

$$transition(s, e) = (p \rightarrow \bar{s'};\ \bar{s}) : \langle s, e \rangle \tag{5}$$

The transition function is a chain of conditions [1, Sec. 11.2.4]: p tests whether $\langle s, e \rangle$ matches a declared transition, f returns the target state, g tries the next or returns $\bar{s}$ as fallback. The fold is pure replay. Applying the event stream produces the current state without side effects.

Facts as events. Creating a fact fires an event. The event type is the fact type; the event arguments are the role bindings. A transition that declares its trigger as a fact type fires automatically when that fact enters P. During *create* (3), *resolve* and *derive* push facts into P; each produces an event that the machine fold (4) consumes. An entity whose facts satisfy all outgoing transition guards advances from its initial status to its terminal status in a single *SYSTEM* call. External events from a message queue, a webhook, or a peer enter the same stream without requiring a local fact. The machine does not distinguish between fact-fired and externally fired events; both are elements of E.

Deletion is a transition to a terminal state. The entity reaches a state with no outgoing transitions and is excluded from query results by restriction.

5.2 Platform Binding

The runtime registers functions into *DEFS* via $\downarrow DEFS$. A browser registers rendering functions; a server registers *httpFetch*, *upsert*, *notify*:

$$(\rho\, fact) : render \rightarrow widget$$
$$(\rho\, fact) : httpFetch \rightarrow response$$
$$(\rho\, fact) : upsert \rightarrow D'$$

Runtime functions that interact with external state execute during *resolve* (3), populating cells before *derive* runs. Functions that consume the representation execute after *emit*. This keeps *derive* pure and terminating over P. A single *SYSTEM* serves browser, server, and storage runtimes by varying *DEFS*, not by varying the logic.

The binding is live: when a cell's contents change via $\downarrow$, every ρ-application over that cell re-evaluates. This is streaming. There is no separate pub/sub layer, no event bus, no callback registry. A subscriber is a ρ-application that has not yet been evaluated against the current D. When D changes, it evaluates. A sensor reading, a Kafka message, a derived inference, and a user click are all facts in P or events in E. They enter D through the same $\downarrow$ operator, and every bound ρ-application sees the update through the same mechanism.

5.3 Data Federation

A federated noun is one whose population is resolved by ρ at evaluation time from an external source rather than read from a local cell. The reading "Noun is backed by External System" compiles to a populating function $populate_n$ registered in *DEFS*. During *resolve* (3), $\rho(populate_n)$ fetches from the external source and produces facts:

$$\rho(populate_n) : I \rightarrow \{f_1, \ldots, f_k\} \subseteq P_{OWA}$$

The fetched facts enter P under OWA (absence is unknown, not false). Constraints and derivation rules evaluate over the unified P without distinguishing local from federated facts. A subset constraint can span a local entity and a federated one. No schema translation is required because the fact type declared in the readings is the response schema, and JSON keys map to role bindings through the compiled schema.

The compile step constructs $populate_n$ from readings: the external system's URL, the noun's URI path, authentication headers, and credential references. SSRF validation rejects internal or loopback URLs at compile time, before any fetch can execute. Cell isolation (Definition 2) ensures federated facts are cached for the request duration so that two concurrent μ applications do not re-fetch.

The approach requires no ETL, no integration middleware, and no separate query language for external data. The external system is a ρ-application, and the rest of the algebra does not change.

5.4 State and Cells

The state D is a sequence of cells $\langle CELL, n, c\rangle$ with Backus's fetch ($\uparrow n : D \rightarrow c$) and store ($\downarrow n : \langle x, D\rangle \rightarrow D'$) operators [1, Sec. 14.3]. *FILE* is a cell whose contents is P. RMAP [5, Ch. 10] determines which facts belong to which cell from the schema's uniqueness constraints: the result is a 3NF row, the complete set of facts that depend on one entity's key. Each entity is a cell. Cells may contain sub-stores [1, Sec. 14.7].

Definition 2 (Cell Isolation)**.** For each cell $\langle CELL, n, c\rangle$ in D, at most one μ application that writes to n may be in progress at any time. Concurrent μ applications over disjoint cells are permitted.

RMAP assigns each entity to its own cell whose fold is independent. A single-threaded runtime satisfies Definition 2 trivially. A distributed system requires one writer per cell. Without isolation, two concurrent *create* operations on the same cell may both pass *validate* against stale P and produce inconsistent state without an error.

6 Formal Foundations

6.1 Domain Primitives

Each combining form [1, Sec. 11.2.4] is an FFP object with a domain interpretation:

FFP Object	Form	Domain
$\langle CONS, s_1, \ldots, s_n\rangle$	Construction	Fact Type
s_i	Selector	Role
$\langle COMP, f, g\rangle$	Composition	Derivation
$\langle COND, p, f, g\rangle$	Condition	Guard
$\langle \alpha, f\rangle$	Apply-to-all	Traversal
$\langle /, f\rangle$	Insert	Aggregation
$\langle bu, f, x\rangle$	Binary-to-unary	Query
$\bar{x}$	Constant	Literal
distl	Distribute-left	Join preparation
eq	Equality test	Predicate
compact	Remove $\perp$ elements	Set formation

Table 1: FFP primitives with domain interpretations.

A fact type $\langle CONS, s_1, \ldots, s_n\rangle$ is a function that produces a fact when applied to instances. Partially applied via *bu* it produces a query.

Two derived forms:

$$Filter(p) \equiv compact \circ \alpha\,(p \rightarrow id;\ \perp) \tag{6}$$

α applies the condition to each element, yielding $\perp$ where p does not hold. *compact* removes $\perp$ elements, producing a proper subsequence. FFP systems choose their primitive set [1, Sec. 11.4.4]; *compact* is an AREST primitive analogous to APL compress.

$$\begin{aligned} foldl\ f\ z\ \langle\rangle &= z \\ foldl\ f\ z\ \langle e, E'\rangle &= foldl\ f\ (f\ z\ e)\ E' \end{aligned} \tag{7}$$

foldl is derivable from Backus's *while* [1, Sec. 11.2.4].

The compiled form for each FORML 2 construct:

FORML 2 Construct	Compiled FFP Object
Entity type `X(.ref)`	$\langle CONS, s_1, \ldots, s_n \rangle$
Fact type `X r Y`	$\langle CONS, s_1, s_2 \rangle$ (binary)
Derivation rule	$\langle COMP, f, g \rangle$
State machine	*foldl transition* s_0 E (7)
Instance fact	$\bar{x}$ asserted into P
Constraints (all compile to Filter(p) : P (6))	
Cardinality family: $p = count(scope) \notin [k, m]$	
UC, MC, FC, XC, XO, OR	scope $\in$ {role, participation}
Membership family: $p = match(target, polarity)$	
IR, AS, SY, AT (self-ref)	target = same population, reversed
IT, TR, AC (path)	target = transitive closure step
SS, EQ (set comparison)	target = cross-fact-type tuples
VC (value)	target = declared value set

Every ORM 2 constraint compiles to a restriction (10) whose predicate belongs to one of two families. The *cardinality* family checks whether a count falls within bounds $[k, m]$. The *membership* family checks whether a tuple exists (or is absent) in a target set. Polarity is the only parameter distinguishing symmetric from asymmetric, subset from exclusion.

6.2 Relational Algebra

Codd's adequate θ_1 [2, Sec. 2.2] as FFP definitions:

$$\pi_L(R) \equiv \alpha\,[s_{i_1}, \ldots, s_{i_k}] : R \tag{8}$$

$$R * S \equiv \mathit{Filter}(eq \circ [s_{sh}]) \circ \mathit{distl} \tag{9}$$

$$R_{L|_M} S \equiv \mathit{Filter}(p) : R \tag{10}$$

$$R \cdot S \equiv \pi_{1s}(R * S) \tag{11}$$

$$\gamma(R) \equiv \mathit{Filter}(eq \circ [s_1, s_n]) : R \tag{12}$$

where s_{sh} selects the shared roles. P is a named set of relations in normal form (ORM 2 elementary facts are 5NF by construction [5, Ch. 12]). Halpin shows that θ_1 = {projection, natural join, restriction, tie} is adequate for the named set of ORM 2 elementary facts [5, Ch. 12]. The combining forms that query the population are the same combining forms that evaluate constraints and derive facts. There is no separate query language.

6.3 Execution Semantics

μ evaluates by applicative-order β-reduction [1, Sec. 13.3.1], repeatedly replacing $(x : y)$ with $(\rho\, x) : y$ under definitions D. Three constraints govern execution:

1. **Reduction order:** applicative (call-by-value).
2. **Scope:** confined to the request-local partition of D determined by RMAP.
3. **Derivation:** forward chaining, monotonic, evaluated to the least fixed point per request. Derivation only adds facts over finite P; the fixed point exists by Knaster-Tarski and is reached in $\leq |P_{\max}| - |P|$ steps. The derivation chain is recorded and available on demand.

The system operates over facts (P), computations (O via ρ), and inputs (I). Time is an input fact, not hidden state. Effects are directives in the output. Identity follows from the reference scheme; *resolve* (3) applies it. The world assumption per noun [5, Ch. 7] governs incompleteness: under CWA, a fact not in P is false; under OWA, it is unknown.

Remark (World assumption on populating functions). All populating functions are runtime functions registered in *DEFS*. A function that exhaustively enumerates its domain operates under CWA. One that cannot operates under OWA. Deontic constraints over OWA-populated facts are sound but not complete: a reported violation is genuine, but the absence of a violation is not a guarantee.

7 Properties

The five theorems form a chain. Each depends on the previous one and eliminates one source of failure in the path from domain knowledge to running software.

Theorem 1 (Grammar Unambiguity)**.** Let N be a declared set of object type names and F a declared set of predicate readings, such that no element of N contains a formal item of the grammar as a substring. Then each FORML 2 sentence over (N, F) has exactly one parse.

Proof. FORML 2 sentences fall into three structural families, each determined by its prefix:

(a) Quantified constraints. Structure: [*modal*] $quant_1$ n_1 r $quant_2$ n_2 [*qual*] where *modal* $\in$ {"It is obligatory that", "It is forbidden that", "It is permitted that", "It is impossible that", "It is possible that", ε}; $quant_1 \in$ {"Each", "For each", "the same", "No", ε}; $quant_2 \in$ {"at most one", "some", "exactly one", "at least k and at most m", "more than one", "no", ε}. Noun positions are resolved by longest-first matching. Each structural position admits exactly one parse because (i) modal prefixes are pairwise non-overlapping, (ii) quantifier phrases are pairwise distinct, (iii) no $n \in N$ contains a formal item as a substring, and (iv) longest-first matching is deterministic. The constraint kind is determined by $(quant_1, quant_2)$.

(b) Conditional constraints. The "If... then" frame splits unambiguously on "then". Ring constraints are identified by all noun bases being the same type. Subset constraints by existential "some" in the antecedent and anaphoric "that" in the consequent. Derivation rules occupy the remainder.

(c) Multi-clause constraints. Keyword-delimited: "if and only if" (EQ), "exactly one of the following holds" (XO), "at most one of the following holds" (XC), "at least one of the following holds" (OR). The keyword is unambiguous.

The three families are distinguished by their sentence prefix at position (1). Within each family, the parse is deterministic. The reference implementation [7] constructively demonstrates unambiguous round-trip parsing for all 17 constraint kinds. □

The hypothesis on N is enforced by the compiler, which rejects any declared name containing a formal item of the grammar as a substring.

Theorem 2 (Specification Equivalence)**.** Let $parse : R \to \Phi$ and $compile : \Phi \to O$ be injective, with R the set of FORML 2 readings, Φ the set of FOL formulas, and O the set of FFP objects. Then:

$$parse^{-1} \circ compile^{-1} \circ compile \circ parse = id_R \tag{13}$$

and $\rho(compile(parse(r)))$ is the executable for every $r \in R$.

Proof. *parse* is injective by Theorem 1. *compile* is injective: each constraint kind compiles to a restriction (10) with a predicate from one of two families parameterized by distinct tuples. Both have left inverses ($compile^{-1}$ is Halpin and Curland's verbalization algorithm [6]); composing yields id_R. Executability: ρ maps the compiled object to a function by [1, Sec. 13.4]. □

Remark (Surjectivity). *compile* is not surjective. Runtime-registered functions occupy the complement. FORML 2 readings cover the domain layer; registered functions cover the platform layer. Together they span *DEFS*.

Theorem 3 (Completeness of State Transfer)**.** Let $F_S : 2^P \to 2^P$ be the monotonic operator that applies all derivation rules in schema S to a population, and let C_S be the set of constraint objects in S. Under Definition 2, *create* (3) applied to input I and state D produces $\langle o, D' \rangle$ such that:

1. $P' = P \cup resolve(S, I)$ contains all entity facts entailed by (S, I);
2. $P'' = lfp(F_S, P')$, the least fixed point of F_S above P';
3. $V = \bigcup\{(\rho\, c) : P'' \mid c \in C_S\}$ collects all violations;
4. o is the REST representation of $(P'', V, links(s))$ and $D' = D[FILE \mapsto P'']$ if V contains no alethic violation, else $D' = D$.

Proof. P is finite. F_S is monotonic. By Knaster-Tarski, $lfp(F_S, P')$ exists and is unique. *validate* applies every $c \in C_S$ as a restriction over the fixed point. *emit* constructs o from P'', V, and $links(s)$ (Theorem 4). Definition 2 ensures no concurrent write to the same cell. □

Theorem 4 (HATEOAS as Projection)**.** All links in the representation are θ_1 operations on P and S.

(a) Transition links. Let $T = \{(s_i, s_j, e_k)\} \subseteq P$ be transition facts and s the current status. Then:

$$links(s) = \pi_{event}(Filter(p) : T) \tag{14}$$

where $p(t) = s_{from}(t) \in \{s\} \cup supertypes(s)$. Transition graphs may contain cycles. It is obligatory that each cycle has some exit transition, a deontic constraint that ensures liveness without requiring structural acyclicity.

(b) Navigation links. Let F be the set of all fact types in S and let U_f denote the role set spanned by the UC on fact type f. Let $S_f = R_f \setminus U_f$ be the non-spanned roles. For a noun n participating in f:

1. **Related collection.** f is a related collection on n, filtered by n. Always applies.
2. **Parent/child.** If $S_f \neq \emptyset$, nouns in S_f are children of nouns in U_f.
3. **Peer resource.** If $S_f = \emptyset$ (spanning UC), f is a navigable peer resource.

For the binary case:

$$children(n) = \pi_{noun(r_c)}\big(Filter(uc(r_c) \wedge eq \circ [noun(\bar{r}_c), \bar{n}]) : F_B\big) \tag{15}$$

$$parent(n) = \pi_{noun(\bar{r}_c)}\big(Filter(uc(r_c) \wedge eq \circ [noun(r_c), \bar{n}]) : F_B\big) \tag{16}$$

$$nav(e, n) = children(n) \cup parent(n) \cup related(n) \cup peers(n) \tag{17}$$

The full link set:

$$links_{full}(e, n, s) = nav(e, n) \cup links(s) \tag{18}$$

Proof. *(a)* $T \subseteq P$. *Filter* is restriction (10); π is projection (8). Both are θ_1. *(b)* Each component of *nav* is a projection over a restriction. The predicate composes *eq* with selector and constant, all primitives from Table 1. All components of $links_{full}$ are θ_1 operations on S and P. □

Theorem 5 (Derivability)**.** Define the *representation* of an entity e in population P under schema S as:

$$\begin{aligned} repr(e, P, S) = {} & \{(\rho\, s_i) : f \mid f \in P,\ f \text{ involves } e\} \\ & \cup \{(\rho\, r) : P \mid r \in S_{derivations}\} \\ & \cup \{(\rho\, c) : P \mid c \in S_{constraints}\} \\ & \cup\ links_{full}(e, n, status(e, P)) \end{aligned}$$

Then for every $v \in repr(e, P, S)$, there exists $f \in O$ such that $v = (\rho\, f) : P$.

Proof. Each component is a ρ-application by construction. Selectors extract attributes. Derivation rules compute derived facts. Constraints produce violation sets via restriction. Links are θ_1 operations (Theorem 4). No value in *repr* is produced outside ρ. □

Corollary 1 (Violation Verbalization)**.** If $(\rho\, c) : P'' \neq \emptyset$ for constraint c compiled from reading r, the violation output is $compile^{-1}(c)$, recovering r by Theorem 2. The error message is the original reading.

Halpin and Curland [6] give the automated verbalization algorithm. Composed θ_1 operations verbalize via FORML 2 qualified readings, which parse unambiguously by Theorem 1.

Corollary 2 (Deletion as Terminal State)**.** A state with no outgoing transitions produces $links(s) = \emptyset$ by Theorem 4. The entity is excluded from query results by $Filter(p_{live}) : P$.

Corollary 3 (Closure Under Self-Modification)**.** After applying *SYSTEM* with operation $compile \circ parse$ to readings R', Theorems 1–5 hold for all subsequent applications over D'.

Proof. Theorem 1 depends on the grammar, not on D. Theorem 2 depends on injective stateless functions. New definitions enter *DEFS* via $\downarrow DEFS$. Theorems 3–5 operate over P and S, both of which include the new content. No theorem's proof depends on D being fixed at initialization. □

Corollary 4 (Constraint Consensus)**.** For two peers sharing population P, one proposes event e and the other applies *validate* (3) to $P \cup resolve(S, e)$. The event is accepted iff V contains no alethic violation. The constraint set C_S is the consensus predicate.

For $n > 2$ peers, a total ordering over the shared event log is additionally required. Safety follows from deterministic replay; liveness from the single-writer cell (Definition 2).

8 Distributed Evaluation

Each cell stores the 3NF row. The event stream is demultiplexed into per-cell streams by RMAP:

$$E_n = Filter(eq \circ [RMAP, \bar{n}]) : E \tag{19}$$

Each cell folds its stream independently:

$$D'_n = foldl\ \mu_n\ D_n\ E_n \tag{20}$$

where μ_n is *SYSTEM* restricted to cell n. Cross-cell queries read committed state:

$$P = \bigcup_n \uparrow FILE : D_n \tag{21}$$

Adding cells to the cluster scales *SYSTEM* horizontally without changing its definition.

A browser runtime is a node in the same model. Cells replicated locally fold events with zero latency, syncing asynchronously. Because the fold is deterministic, two nodes folding the same events reach the same state. Local constraints (cardinality, value) evaluate immediately. Cross-cell constraints (subset, external uniqueness) evaluate against committed state after sync. Local evaluation is optimistic feedback, not a security boundary.

For anonymous peers, events carry cryptographic signatures for identity, schema modifications require authorization (a deontic constraint), and every peer validates independently. The hash chain provides ordering, constraints provide validity, signatures provide identity, and authorization provides permission, all composed from the existing formalism.

9 Middleware Elimination

Traditional middleware (authentication, validation, rate limiting, transformation) is either a restriction over P, a composition, or a fact in P. Authorization is a derivation: "User accesses Domain iff User belongs to Organization that manages Domain" compiles to $\langle COMP, f, g \rangle$ whose ρ-image computes the access predicate over P. External service calls, database persistence, and UI rendering are runtime functions registered into *DEFS* and applied via ρ. Theorem 5 leaves no room for a computation layer outside ρ.

10 Conclusion

The five theorems form a dependency chain from unambiguous grammar to full derivability, each eliminating one source of failure between domain knowledge and running application. Self-modification preserves all five (Corollary 3). Fielding's remaining architectural constraints follow: statelessness holds because each application of *SYSTEM* is a pure transition on D; cacheability because the representation is a deterministic function of P and S (Theorem 5); layered system because cells are location-independent; code-on-demand because self-modification via ingesting readings preserves the theorems.

The five properties compose into a guarantee for automated agents. An LLM writing FORML 2 readings produces unambiguous specifications (Theorem 1) that are the executables (Theorem 2). An LLM consuming the API reads values that are ρ-applications over P (Theorem 5) and discovers every valid action via HATEOAS links (Theorem 4). There are no undocumented endpoints, no hidden middleware, no informal contracts to hallucinate through. The forward chainer evaluates derivation rules to the least fixed point (Theorem 3), and the LLM reads conclusions, not premises. The logical chain was evaluated deterministically by the engine. When the chain itself is needed, the derivation trace is available on demand.

The algebraic laws of FP [1, Sec. 12.2] are equations between functional forms. Since FFP objects represent the same forms via ρ, an FFP object can be transformed to an equivalent object by applying these laws. Ahead-of-time compilation from objects to optimized evaluation trees is one such transformation.

Common operational concerns map directly to the formalism: versioning is the event stream, migration is ingestion of new readings, backward compatibility is preserved by construction, and auditability is the derivation trace.

The gap between domain knowledge and running software is the gap where requirements drift, implementations diverge, and production breaks. AREST eliminates the gap by identifying what was never separate to begin with.

Acknowledgments

Claude (Anthropic) was used as a programming assistant during implementation and as a drafting aid during manuscript preparation. All architectural decisions and the composition of FFP/AST with REST were made by the human author.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Commun. ACM*, 21(8):613–641, 1978.

[2] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Commun. ACM*, 13(6):377–387, 1970.

[3] E.F. Codd, "Relational Completeness of Data Base Sublanguages," in R. Rustin, ed., *Data Base Systems*, Prentice-Hall, pp. 65–98, 1972.

[4] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[5] T. Halpin, *Information Modeling and Relational Databases*, Morgan Kaufmann, 2001.

[6] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[7] T. Halpin and M. Curland, NORMA (Natural Object-Role Modeling Architect), open-source reference implementation, `https://github.com/ormsolutions/NORMA`.